

CSCI 112

Programming with C

LECTURE: 16

J. L. Pach

OUTLINE:

- Binary file I/O: `fread()`, `fwrite()`, `fseek()`, `ftell()`, `rewind()`
- Text file I/O: `fgetc()`, `fgets()`, `fscanf()`, `fputc()`, `fputs()`, `fprintf()`
- Text vs binary mode – text files in binary mode

TEXT FILE I/O FUNCTIONS

- **For reading text from file**:

  `fgetc()`: Reads a single character from a file.

  `fgets()`: Reads a line from a file.

  `fscanf()`: Reads formatted input from a file (similar to `scanf()`).

- **For writing text for file**:

  `fputc()`: Writes a single character to a file.

  `fputs()`: Writes a string to a file.

  `fprintf()`: Prints formatted output to a file (similar to `printf()`).

Reading Text from a file

1. `int fgetc(FILE *stream)`
   i. **Description**: Reads a single character from the specified file stream.
   ii. **Return Type**: Returns the character read as an unsigned char cast to an int, or `EOF` on end of file or error.
   iii. **Example**: `int ch = fgetc(file_pointer);`
2. `char *fgets(char *str, int n, FILE *stream)`
   i. **Description**: Reads a line from the file and stores it in the character array str.
   ii. **Return Type**: Returns str on success, or `NULL` on error or when end of file occurs.
   iii. **Example**: `char *result = fgets(buffer, sizeof(buffer), file_pointer);`
3. `int fscanf(FILE *stream, const char *format, ...)`
   i. **Description**: Reads formatted input from a file based on the format string, similar to `scanf`.
   ii. **Return Type**: Returns the number of items successfully read, or `EOF` if an error or end of file occurs before any items are matched.

Writing Text to a file

1. `int fputc(int char, FILE *stream)`
   i. **Description**: Writes a single character to the specified file stream.
   ii. **Return Type**: Returns the written character as an unsigned char cast to an int, or EOF on error.
   iii. **Example**: `int result = fputc('A', file_pointer);`
2. `int fputs(const char *str, FILE *stream)`
   i. **Description**: Writes a string to the file.
   ii. **Return Type**: Returns a non-negative number on success, or EOF on error.
   iii. **Example**: `int result = fputs("Hello, World!\n", file_pointer);`
3. `int fprintf(FILE *stream, const char *format, ...)`
   i. **Description**: Writes formatted output to the specified file stream, similar to printf().
   ii. **Return Type**: Returns the number of characters written, or a negative value if an error occurs.
   iii. **Example**: `int chars_written = fprintf(file_pointer, "Integer: %d, Float: %f\n", int_var, float_var);`

Example – fputc() and fputs()

```
FILE *file = fopen("hello.txt", "wt");
if(file)
{
    fputc('H', file);
    fputc('i', file);
    fputc('\n', file);

    fputs("C is fun!\n", file);
    fputs("Line 3", file);
    fclose(file);
}
else
    printf("Error");
```

```
hello.txt:
Hi
C is fun!
Line 3
```

⚒ Unlike `puts()`, the function `fputs()` does **not** add `'\n'` – you must write it yourself.

Example – fprintf()

```
const char *names[] = {"Anna", "Piotr", "Ewa"};
float scores[] = {4.5f, 3.0f, 5.0f};

FILE *file = fopen("grades.txt", "wt");
if(file)
{
    for(int i = 0; i < 3; i++)
        fprintf(file, "%s %.1f\n",
                names[i], scores[i]);
    fclose(file);
}
else
    printf("Error");
```

```
grades.txt:
Anna 4.5
Piotr 3.0
Ewa 5.0
```

⚒ `fprintf()` works exactly like `printf()` – only the first argument is new. In fact `printf(...)` is `fprintf(stdout, ...)`.

Example – mode `w` vs mode `a`

```c
/* run the program 2 times */
FILE *f = fopen("log.txt", "wt");
if(f)
{
    fprintf(f, "Program started\n");
    fclose(f);
}
```

log.txt:
Program started

```c
/* run the program 2 times */
FILE *f = fopen("log.txt", "at");
if(f)
{
    fprintf(f, "Program started\n");
    fclose(f);
}
```

log.txt:
Program started
Program started

Example – fgetc()

```
FILE *file = fopen("hello.txt", "rt");
if(file)
{
    int ch;
    int chars = 0, lines = 0;
    while((ch = fgetc(file)) != EOF)
    {
        putchar(ch);
        chars++;
        if(ch == '\n')
            lines++;
    }
    fclose(file);
    printf("\n%d chars, %d lines\n",
           chars, lines);
}
else
    printf("Error");
```

```
Result:
Hi
C is fun!
Line 3
19 chars, 2 lines
```

⚒ We count `'\n'` characters – the last line has no `'\n'`, so 3 lines of text give `lines = 2`.

`EOF` and end-of-file detection

WRONG

```
char ch;              /* wrong type    */
while(!feof(file))    /* checked too late */
{
   ch = fgetc(file);
   putchar(ch);
}
/* prints one extra garbage character */
```

CORRECT

```
int ch;               /* int, not char */
while((ch = fgetc(file)) != EOF)
{
   putchar(ch);
}
/* afterwards: feof() or ferror()
   tell us WHY the loop has ended   */
```

- `EOF` is a symbolic constant defined in `<stdio.h>`; its value is a negative integer, typically `-1`.
- `fgetc()` returns `int`, so `EOF` can never be confused with a real character (0–255).

Example – fgets()

```c
#include <string.h>
/* ... */
FILE *file = fopen("grades.txt", "rt");
if(file)
{
    char line[50];
    int no = 1;
    while(fgets(line, sizeof(line), file))
    {
        line[strcspn(line, "\n")] = '\0';
        printf("%d: [%s]\n", no++, line);
    }
    fclose(file);
}
else
    printf("Error");
```

Result:

```
1: [Anna 4.5]
2: [Piotr 3.0]
3: [Ewa 5.0]
```

⚒ `fgets()` reads at most `n-1` characters, **keeps** the `'\n'` and always adds `'\0'`. `strcspn()` removes the `'\n'`.

Example – fscanf()

```
FILE *file = fopen("grades.txt", "rt");
if(file)
{
    char name[20];
    float score, sum = 0;
    int count = 0;
    while(fscanf(file, "%19s %f",
                 name, &score) == 2)
    {
        printf("%s -> %.1f\n", name, score);
        sum += score;
        count++;
    }
    fclose(file);
    if(count > 0)
        printf("Average: %.2f\n", sum / count);
}
else
    printf("Error");
```

```
Result:
Anna -> 4.5
Piotr -> 3.0
Ewa -> 5.0
Average: 4.17
```

⚒ We compare the result with the number of expected items (`== 2`), not with `EOF` .

Text I/O functions – summary

Function	Unit of data	Signals end of file / error
`fgetc()`	one character	returns `EOF`
`fgets()`	one line (string)	returns `NULL`
`fscanf()`	formatted values	returns fewer items than expected (or `EOF`)
`fputc()`	one character	returns `EOF` on error
`fputs()`	string	returns `EOF` on error
`fprintf()`	formatted values	returns a negative value on error

Text mode vs binary mode

- A text file is **only a sequence of bytes**. The letters `t` and `b` in `fopen()` change one thing: the translation of the end of line.

Platform	`'\n'` written in `"wt"`	`'\n'` written in `"wb"`
Linux / macOS	`0A`	`0A`
Windows	`0D 0A` (CR LF)	`0A`

- When reading in `"rt"` on Windows, `0D 0A` is translated back to `'\n'`; in `"rb"` the program sees both bytes.
- In `"rt"` on Windows, the byte `0x1A` (Ctrl+Z) may be treated as the end of file.

Example – '\n' in text and in binary mode

```
FILE *t = fopen("t.txt", "wt");
FILE *b = fopen("b.txt", "wb");
if(t && b)
{
    fprintf(t, "A\nB\n");
    fprintf(b, "A\nB\n");
}
if(t) fclose(t);
if(b) fclose(b);
```

```
Size of the files:
Linux:     t.txt = 4, b.txt = 4
Windows:   t.txt = 6, b.txt = 4

b.txt:  41 0A 42 0A
t.txt:  41 0D 0A 42 0D 0A
        (Windows)
```

- `fprintf()` and `fputs()` also work on a file opened with `b` – the mode only controls the translation of `'\n'`.

Example – writing a text file in binary mode

```c
#include <string.h>
/* ... */
const char *msg = "Hello, binary mode!\n";

FILE *file = fopen("msg.txt", "wb");
if(file)
{
    fwrite(msg, sizeof(char), strlen(msg), file);
    fprintf(file, "x = %d\n", 42);
    fclose(file);
}
else
    printf("Error");
```

msg.txt (opens in any text editor):

```
Hello, binary mode!
x = 42
```

⚒ We used `strlen(msg)`, not `sizeof(msg)` – the `'\0'` is **not** written. Compare with the `"Some text"` example, where all 10 bytes went to the file.

Example – reading a text file in binary mode

```
FILE *file = fopen("msg.txt", "rb");
if(file)
{
    char buffer[8];
    size_t n;
    while((n = fread(buffer, 1,
                     sizeof(buffer), file)) > 0)
    {
        fwrite(buffer, 1, n, stdout);
        printf("|");
    }
    fclose(file);
}
else
    printf("Error");
```

```
Result:
Hello, b|inary mo|de!
x = |42
|
```

⚒ `fread()` returns the number of elements read. The last block is shorter (3 bytes) – that is why we use `n`, not `sizeof(buffer)`.

Example – the whole text file in memory

```
FILE *file = fopen("msg.txt", "rb");
if(file)
{
    char content[100];

    fseek(file, 0, SEEK_END);
    long size = ftell(file);
    rewind(file);

    if(size < (long)sizeof(content))
    {
        fread(content, 1, size, file);
        content[size] = '\0';
        printf("Size: %ld bytes\n", size);
        printf("%s", content);
    }
    fclose(file);
}
else
    printf("Error");
```

```
Result:
Size: 27 bytes
Hello, binary mode!
x = 42
```

⚒ `fseek()` + `ftell()` + `rewind()` give the exact size of the file. In `"rb"` it is the number of bytes – in `"rt"` on Windows the size could differ from the number of characters read.

Example – hex dump of a text file

```
FILE *file = fopen("hello.txt", "rb");
if(file)
{
    int ch;
    long pos = 0;
    while((ch = fgetc(file)) != EOF)
    {
        printf("%02X ", ch);
        if(++pos % 8 == 0)
            printf("\n");
    }
    fclose(file);
}
else
    printf("Error");
```

```
Result:
48 69 0A 43 20 69 73 20
66 75 6E 21 0A 4C 69 6E
65 20 33
```

- `H` = `48`, `i` = `69`, `'\n'` = `0A`, `' '` = `20` … A text file is just bytes; `fgetc()` works in binary mode too.

Example – copying any file (text or binary)

```
FILE *src = fopen("hello.txt", "rb");
FILE *dst = fopen("copy.txt", "wb");
if(src && dst)
{
    char buffer[256];
    size_t n;
    while((n = fread(buffer, 1,
                sizeof(buffer), src)) > 0)
        fwrite(buffer, 1, n, dst);
    printf("Copy done\n");
}
else
    printf("Error");

if(src) fclose(src);
if(dst) fclose(dst);
```

- Binary mode means **no translation** – the copy is identical, byte by byte.
- The same code copies a `.txt`, a `.png` or a `.pdf` file.
- Copying a binary file in text mode could change or cut the data (`0x1A`, `0D 0A`).

Example – random access in a text file

```c
#define RECORD 10   /* 9 chars + '\n' */

FILE *file = fopen("names.txt", "rb");
if(file)
{
   char line[RECORD + 1];
   int n = 2;      /* 0-based index */

   fseek(file, (long)n * RECORD, SEEK_SET);
   if(fread(line, 1, RECORD, file) == RECORD)
   {
      line[RECORD] = '\0';
      printf("Record %d: %s", n, line);
   }
   fclose(file);
}
else
   printf("Error");
```

```
names.txt (every line = 10 bytes):
Anna     \n
Piotr    \n
Ewa      \n
Zofia    \n

Result:
Record 2: Ewa
```

⚒ This works only if every line has the same length in bytes – the offset is `n * RECORD`. We open with `"rb"`, so `'\r'` can't shift the offsets.

Key takeaways

- Always check the result of `fopen()` and always call `fclose()`.
- Text functions: `fgetc()` / `fputc()`, `fgets()` / `fputs()`, `fscanf()` / `fprintf()`.
- Read with `int ch` and `!= EOF` – not with `while(!feof())`.
- Modes `t` and `b` differ only in the translation of `'\n'` (and `0x1A`).
- A text file can be processed in binary mode (`fread()`, `fwrite()`, `fseek()`, `ftell()`) – useful for copying, size, block reading and random access.

THANK

You!